Electrical Engineering Core Labs

Indian Institute of Technology Hyderabad (IIT Hyderabad)

SUMMER INTERNSHIP REPORT

Learning Digital Design through
phone-based Arduino Embedded Systems

May 23 – July 15, 2026

Submitted by

Ayaan Syed
Jamia Millia Islamia, New Delhi

Under the guidance of

Prof. G.V.V.
Department of Electrical Engineering,
IIT Hyderabad

Declaration: This report is a true account of the work carried out during the summer internship at the Electrical Engineering Core Labs, IIT Hyderabad, under the guidance of Prof. G.V.V. All sources and references have been duly acknowledged.

Contents

1 Introduction **1**
1.1 About the Host Institute 1

2 Objectives of the Internship **2**

3 Internship Timeline **3**

4 Tools, Software and Setup **4**

5 Methodology: The Three-Step Development Workflow **6**

6 Foundational Tasks **7**
6.1 Seven-Segment Display 7
6.2 IC 7447 – BCD to Seven-Segment Decoder 7
6.3 IC 7474 – D Flip-Flop / Decade Counter 8

7 Gate Problem: GATE EE 2019, Q44 **9**

8 LaTeX Documentation of Question Paper **10**

9 Minor Project: Digital Clock **11**
9.1 Design Approach 11
9.2 Breadboard Prototype 11
9.3 Bootloader Burning and Standalone ATmega328P Programming 12
9.4 Standalone Testing 12
9.5 Perfboard Assembly and Soldering 13
9.6 Final Testing and Validation 14

10 Digital Electronics **15**

11 Challenges Faced **17**
11.1 Unconventional Development Workflow 17
11.2 Short Circuits and Component Damage 17
11.3 Diagnosing Faults After Soldering 17
11.4 Bootloader and Fuse Configuration 17

12 Learning Outcomes and Reflection **19**

13 Skills Gained **20**

14 Conclusion **21**

A Glossary of Key Terms **22**

B References and Resources **23**

Chapter 1

Introduction

This report presents a summary of the summer internship undertaken at the Electrical Engineering Core Labs, Indian Institute of Technology Hyderabad (IIT Hyderabad), from May 23, 2026 to July 15, 2026, under the guidance of Prof. G.V.V. The internship focused on Arduino programming and embedded systems, with an emphasis on understanding hardware-software interaction at multiple levels — from high-level code down to assembly-level programming — and on building a functional embedded project from the ground up.

Unlike a conventional Arduino IDE-based workflow using a laptop, this internship used an unconventional and highly hands-on approach: programming and uploading code to the Arduino using an Android phone via a USB OTG cable, using Termux as the development environment. This method required setting up a complete Linux-based command-line toolchain on a mobile device, giving deeper insight into how embedded toolchains work under the hood, rather than relying on the abstraction provided by desktop IDEs.

The internship progressed from foundational exercises with basic digital ICs to a substantial minor project — the design, coding, and hardware assembly of a fully functional digital clock, including custom PCB-free assembly on a perfboard.

1.1 About the Host Institute

The Indian Institute of Technology Hyderabad (IIT Hyderabad) is one of India's premier institutions for engineering education and research. Its Electrical Engineering Core Labs support undergraduate and intern learning through hands-on, lab-based courses that emphasize practical circuit design, embedded systems, and microcontroller programming alongside theoretical coursework. The Core Labs are equipped for both breadboard-level prototyping and permanent circuit assembly, making them well suited to project-based internships such as this one.

The internship was carried out under the direct guidance of Prof. G.V.V., whose instructional material — including setup guides and package installation instructions published on GitHub — formed the backbone of the toolchain used throughout the internship.

The internship spanned approximately eight weeks, structured broadly into three phases: an initial orientation and toolchain-setup phase, a foundational-exercises phase covering basic digital ICs, and a major-project phase centered on the design and construction of a standalone digital clock.

Chapter 2

Objectives of the Internship

- To gain hands-on experience with Arduino and embedded systems programming beyond the standard IDE workflow.
- To understand the three levels of embedded development: hardware design, software-level (compiled) code, and assembly-level code.
- To learn to set up and use a Linux-based mobile development environment (Termux) for compiling and uploading code to microcontrollers.
- To gain practical exposure to fundamental digital ICs such as the 7447 (BCD-to-7-segment decoder) and 7474 (D flip-flop / decade counter).
- To design, build, and test a digital clock circuit — from breadboard prototyping to a soldered perfboard implementation.
- To learn bootloader burning and standalone microcontroller programming using an ATmega328P chip.
- To develop practical hardware debugging and soldering skills.

Chapter 3

Internship Timeline

The internship ran for approximately eight weeks, from May 23 to July 15, 2026. The table below gives an approximate phase-wise breakdown of how the internship progressed, from initial environment setup through to the completed digital clock.

Table 3.1: Phase-wise Internship Timeline

Period	Activities
Week 1	Orientation; introduction to the lab, to Prof. G.V.V.'s workflow, and to the overall goals of the internship.
Week 2	Setting up Termux and Linux on the phone and laptop; installing toolchain packages from GitHub instructions; first successful compile-and-upload cycle on the Arduino Uno.
Week 3	Learning and practicing the three-step workflow (hardware, software-level code, assembly-level code) on simple I/O tasks.
Week 4	Foundational tasks: seven-segment display interfacing, and working with the 7447-decoder IC. Also worked on LaTeX documentation of a question paper using Neovim as the editor, including implementing a watermark and other key formatting features.
Week 5	Foundational tasks continued: working with the 7474 D flip-flop and building decade counter circuits; a GATE problem was also implemented on the breadboard using Arduino; transition into planning the digital clock project.
Week 6	Digital clock: breadboard prototyping and testing using the Arduino Uno.
Week 7	Digital clock: bootloader burning and programming of standalone ATmega328P chips; standalone breadboard testing.
Week 8	Digital clock: perfboard soldering, assembly, debugging, and final validation; wrap-up and report preparation.

Chapter 4

Tools, Software and Setup

The internship made use of an unconventional but effective toolchain built entirely around a smartphone, in place of a traditional laptop-based Arduino IDE setup. The core components of this setup are summarized below.

Table 4.1: Core Tools and Components

Component	Details
Microcontroller	Arduino Uno; standalone ATmega328P chips
Development Environment	Termux (Linux terminal emulator for Android)
Upload Interface	Arduino Droid (Android app, using "Upload Precompiled")
Connection	USB OTG cable (phone to Arduino Uno)
Operating System	Linux, installed both within Termux and on the lab laptop
Toolchain Packages	Installed via Termux using package installation instructions provided by Prof. G.V.V. through GitHub
Prototyping Hardware	Breadboard, perfboard, jumper wires, copper wire, soldering iron
ICs Used	7447 (BCD-to-7-segment decoder), 7474 (D flip-flop / decade counter)

Setting up Termux involved installing the Linux command-line environment on the phone and then downloading the required compiler and toolchain packages using setup instructions shared by Prof. G.V.V. on GitHub. This setup effectively turned the phone into a portable embedded development workstation, capable of writing, compiling, and flashing code directly to Arduino hardware without a laptop.

In parallel, a full Linux operating system was also installed on the lab laptop. Having both a phone-based (Termux) and a laptop-based Linux environment available meant that the same command-line toolchain — compilers, assemblers, and AVR upload utilities — could be used consistently across devices, reinforcing the idea that embedded development does not have to be tied to a specific IDE or operating system, but rather to a portable, script-driven toolchain.

This setup stood in deliberate contrast to the conventional Arduino IDE, which hides most of the compilation and upload process behind a single "Upload" button. By instead compiling code manually within Termux, saving the output to the Downloads folder, and uploading it through ArduinoDroid's "Upload Precompiled" option, each stage of the

process — compilation, file generation, and flashing — had to be understood and executed as a distinct, visible step.

Chapter 5

Methodology: The Three-Step Development Workflow

Every task during the internship, regardless of complexity, followed a consistent three-step workflow that mirrored the actual layers of embedded system development:

1. **Hardware Design** — Planning and building the physical circuit, including component selection and wiring on breadboard or perfboard.
2. **Software-Level Code** — Writing the logic in C/C++ within Termux, compiling it on-device, saving the compiled output to the phone's Downloads folder, and uploading it to the Arduino using Arduino Droid's "Upload Precompiled" feature.
3. **Assembly-Level Code** — Writing and compiling equivalent assembly code, generating the corresponding `.hex` file, saving it to Downloads, and uploading it via the same "Upload Precompiled" route in Arduino Droid.

Working through both the software and assembly levels for the same hardware task reinforced a clear understanding of what happens between writing code and it actually running on a microcontroller — including compilation, hex file generation, and how precompiled binaries are flashed onto hardware.

Chapter 6

Foundational Tasks

Before progressing to the minor project, several foundational exercises were completed to build familiarity with the hardware-software-assembly workflow and with basic digital ICs.

6.1 Seven-Segment Display

A basic seven-segment display was interfaced with the Arduino, driven at both the software and assembly level, to understand digit-pattern generation and I/O pin control. At the software level, this meant writing code that mapped each digit (0–9) to the correct combination of segment pins to switch on; at the assembly level, the same mapping had to be implemented through direct register and port manipulation, without the convenience of higher-level variables or functions. Comparing the two implementations for the exact same task made the relationship between high-level code and the underlying register operations much clearer.

- Code: https://github.com/gadepall/fwc-1/blob/main/ide/sevenseg/codes/sevenseg/sevenseg.ino

6.2 IC 7447 – BCD to Seven-Segment Decoder

The 7447-decoder IC was used to convert binary-coded decimal (BCD) input into the signals needed to drive a seven-segment display directly, reducing the number of Arduino pins required for display control. Rather than driving each of the seven display segments individually from the Arduino, only four BCD input lines needed to be controlled, with the 7447 handling the decoding logic in hardware. This task illustrated a recurring theme of the internship: how dedicated ICs can offload logic that would otherwise have to be implemented entirely in software.

- Code: https://github.com/gadepall/fwc-1/blob/main/ide/7447/codes/gvv_ard_7447/gvv_ard_7447.ino
- Code (Increment/Decrement): https://github.com/gadepall/digital-design/blob/main/ide/7447/codes/inc_dec/inc_dec.ino
- Code (Input Increment/Decrement): https://github.com/gadepall/fwc-1/blob/main/ide/7447/codes/ip_inc_dec/ip_inc_dec.ino

6.3 IC 7474 – D Flip-Flop / Decade Counter

The 7474 D flip-flop was used to build and study decade counter behaviour, forming a key building block later reused conceptually in the digital clock's timekeeping logic. We used two 7474 ICs, one 7447 and a 7-segment display. Since each 7474 contains two flip-flops, in total there are four flip-flops. Therefore the number of states is ≤ 16; with the help of the coding logic, a counter of 10 states was realized. Understanding how flip-flops chain together to divide and count clock pulses was directly relevant to the digital clock project, where seconds, minutes, and hours all rely on the same underlying principle of counting and rolling over at fixed intervals.

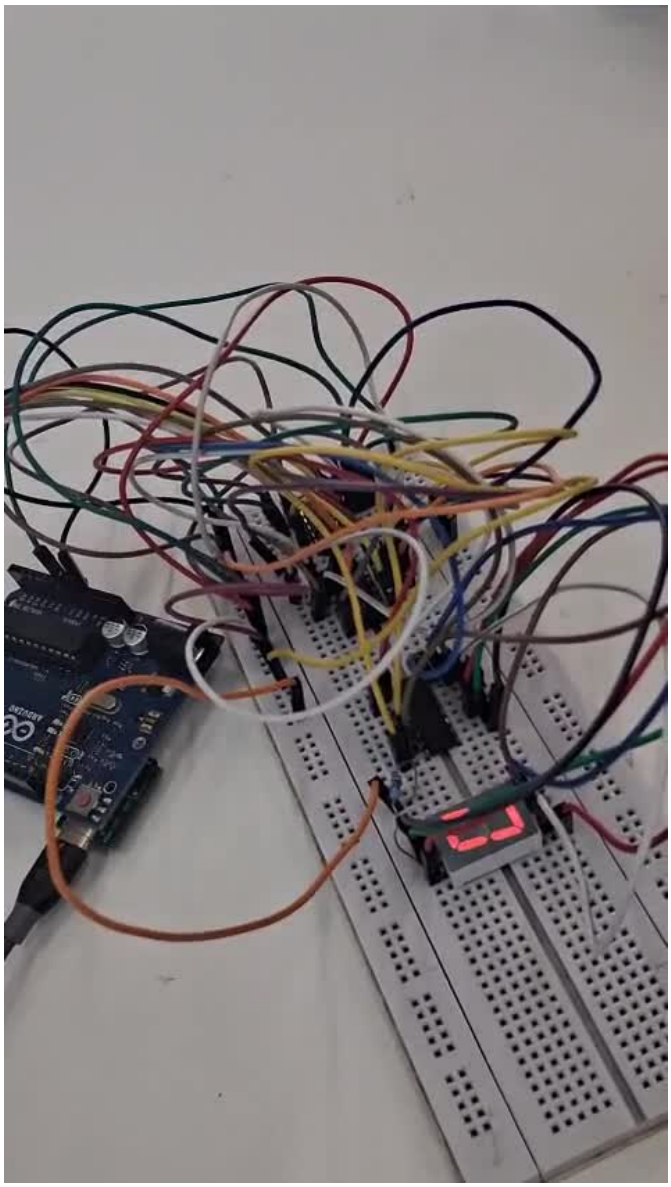


Figure 6.1: Frame from the video demonstration of the 7474-decade counter circuit in operation.

Chapter 7

Gate Problem: GATE EE 2019, Q44

As an extension of the work on basic digital ICs, a GATE-level digital logic problem (GATE EE 2019, Question 44) was also implemented and verified on the breadboard using the Arduino, as part of the Week 5 activities. The question, along with the circuit to be simplified, is reproduced below.

44. In the circuit shown below in Fig. 3.26, X and Y are digital inputs, and Z is a digital output. The equivalent circuit is a (GATE EE 2019)

a) NAND gate
b) NOR gate
c) XOR gate
d) XNOR gate

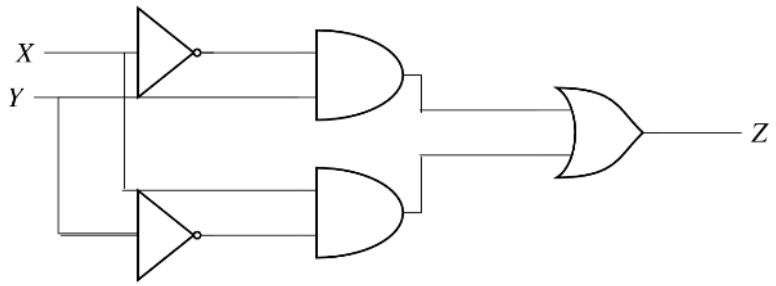


Fig. 3.26

Figure 7.1: GATE EE 2019, Q44 — equivalent logic circuit for digital inputs X, Y and digital output Z.

In the circuit, X and Y are first passed through NOT gates and combined with the original signals using two AND gates, whose outputs are then combined using an OR gate to produce Z. Working through the Boolean expression for this arrangement shows that it reduces to the XOR function of X and Y, making the correct answer option (c) XOR gate. Rebuilding this circuit on the breadboard with discrete NOT, AND, and OR gate ICs, and verifying its truth table against an Arduino-driven XOR gate, confirmed this equivalence in hardware rather than only algebraically.

- Code: https://github.com/ayaansyed8/GATE-Q/blob/main/Gate

Chapter 8

LaTeX Documentation of Question Paper

In Week 4, a GATE-style Computer Science and IT question paper was typeset in LaTeX, edited entirely in Neovim and compiled with `pdflatex`. The document uses a custom header/footer with an institute logo (`fancyhdr`), a faint page watermark (`eso-pic`, `transparent`), custom `\question` and `\mcqtable` macros for consistent formatting across all 65 questions, structured tables (`booktabs`, `tabularx`), and `tikz`-drawn figures.

- Source: `Question_Paper_LaTeX_Source.tex`

Chapter 9

Minor Project: Digital Clock

The major portion of the internship was dedicated to designing, building, and testing a fully functional digital clock — from an Arduino-driven breadboard prototype to a final, independently powered, soldered circuit built around a standalone ATmega328P chip. This project brought together every concept covered in the foundational tasks: digital display driving, counter logic, and the three-step hardware/software/assembly workflow, applied at a larger and more integrated scale.

9.1 Design Approach

The clock was approached in clearly separated stages rather than being built end-to-end in one attempt. Each stage was fully validated before moving to the next: circuit logic on the breadboard first, then standalone chip behaviour, and only then permanent soldered assembly. This staged approach meant that if something failed later on, the number of possible causes to check was much smaller, since the earlier stages had already been confirmed working. This sequencing was itself one of the more valuable practical lessons of the project.

9.2 Breadboard Prototype

The digital clock circuit was first built and tested on a breadboard, with the Arduino Uno used directly to flash and run the clock logic. Building on the breadboard first, rather than soldering immediately, allowed the wiring and component placement to be adjusted freely while the timekeeping logic was being written and debugged. This stage validated the overall circuit design and timing logic before committing to a standalone, soldered implementation, and served as the working reference circuit for every later stage of the project.

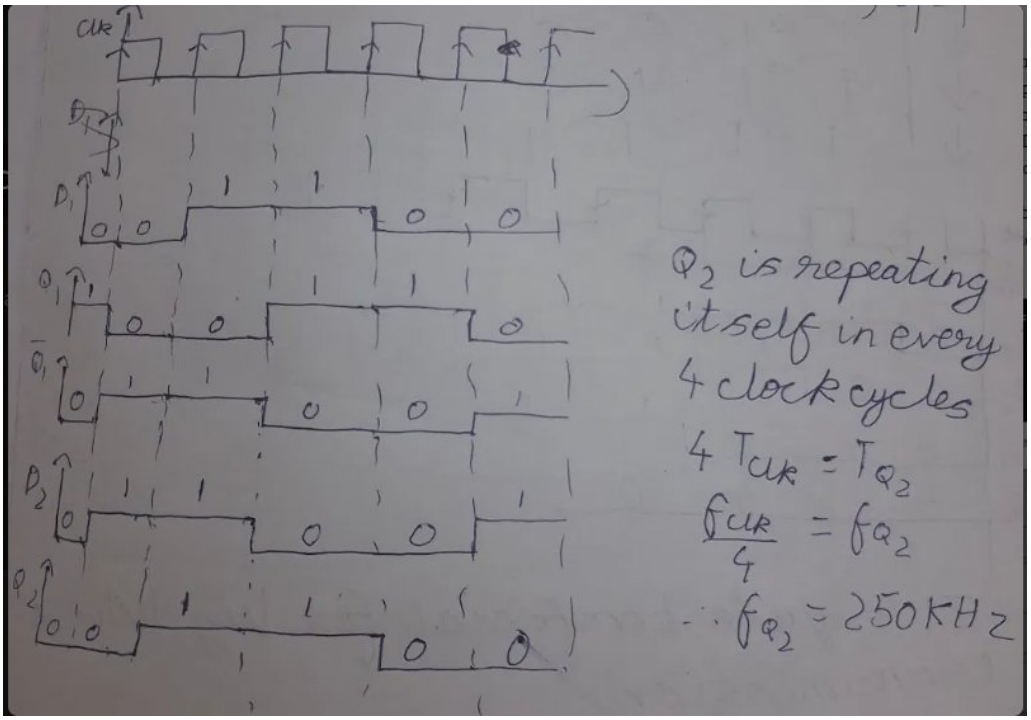


Figure 9.1: Digital clock breadboard prototype with seven-segment displays driven by the Arduino Uno.

9.3 Bootloader Burning and Standalone ATmega328P Programming

To move beyond the Arduino Uno board itself, individual ATmega328P chips were programmed to run standalone — that is, without being mounted on an Arduino Uno board at all. This involved burning the bootloader onto each bare chip, a process that reconfigures the chip's fuse settings so that it can subsequently accept and run uploaded programs on its own, exactly as an Arduino Uno's onboard microcontroller does out of the box.

Once bootloaded, the Arduino Uno itself was repurposed as an in-system programmer (ISP), used to upload the compiled clock code directly onto each individual standalone ATmega328P chip. This step required care, since an incorrect fuse configuration or a failed upload would leave the chip unresponsive and require the bootloading process to be repeated from scratch.

9.4 Standalone Testing

Each programmed standalone ATmega328P chip was then tested on the breadboard circuit, independent of the Arduino Uno board, to confirm that the clock logic ran correctly on the bare chip before moving to permanent assembly. This step was an important checkpoint: any issue caught here on the breadboard was far easier and quicker to fix than the same issue discovered later, after the circuit had already been soldered onto the perfboard.

9.5 Perfboard Assembly and Soldering

Once the standalone circuit was validated, the design was transferred to a perfboard for a permanent build. This involved soldering all components — the standalone ATmega328P, supporting passive components, and the clock display — in place, and manually routing connections using copper wire in place of the Arduino Uno's internal traces and the breadboard's reconfigurable strips.

This stage was significantly more hands-on and unforgiving than breadboard prototyping. On a breadboard, a wrong connection can be pulled out and replaced in seconds; on a soldered perfboard, correcting a mistake means desoldering, which risks damaging components or lifting copper pads. As a result, this phase demanded far greater attention to planning the physical layout before soldering began, and it was also where most of the debugging time during the internship was spent.

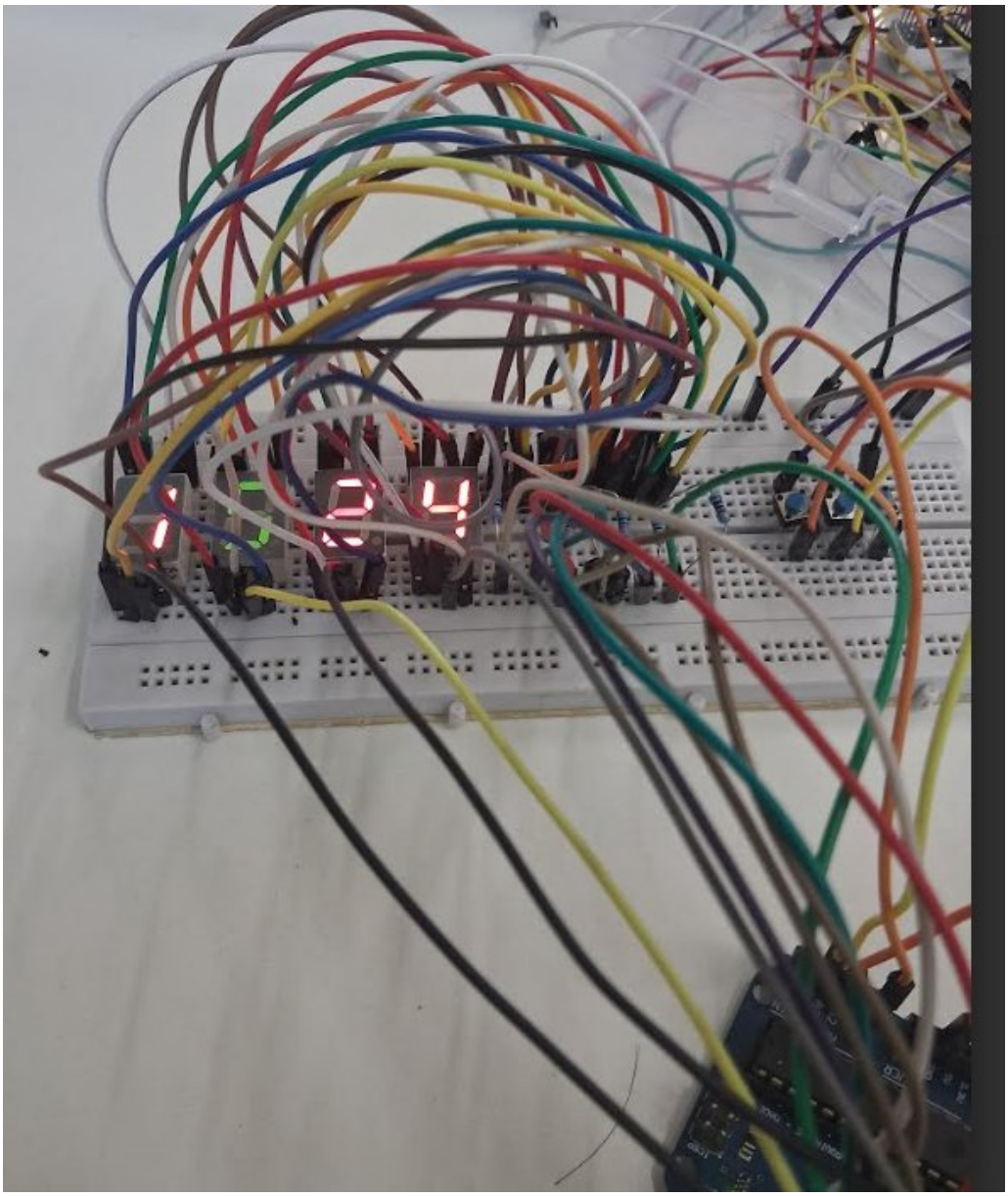


Figure 9.2: Soldered perfboard assembly of the digital clock circuit, with copper-wire connections.

9.6 Final Testing and Validation

After assembly, the soldered digital clock was powered and tested independently of the breadboard prototype and the Arduino Uno, confirming that the standalone board — chip, bootloader, uploaded code, and physical wiring — functioned correctly as a complete, self-contained unit.

Chapter 10

Digital Electronics

Alongside the hardware and documentation work, time was also set aside to strengthen the underlying fundamentals of digital electronics, which proved essential to the rest of the internship. This was done by working through GATE-level questions provided in Prof. G.V.V.'s reference material, while building up the core concepts from Morris Mano's *Digital Design* — connecting the theory of Boolean simplification, Karnaugh maps, and prime implicants directly to the circuits being built on the breadboard.

One representative example is a four-variable minimization problem, solved by constructing the Karnaugh map, identifying the prime implicants and essential prime implicants, and reading off the minimum SOP and POS expressions.

$$F(A,B,C,D) = \Sigma m(0,2,5,7,8,10,13,15) + \Sigma d(1,3,9,11,14)$$

Find:

- Minimum SOP
- Minimum POS

Figure 10.1: GATE-style Boolean minimization question — find the minimum SOP and minimum POS.

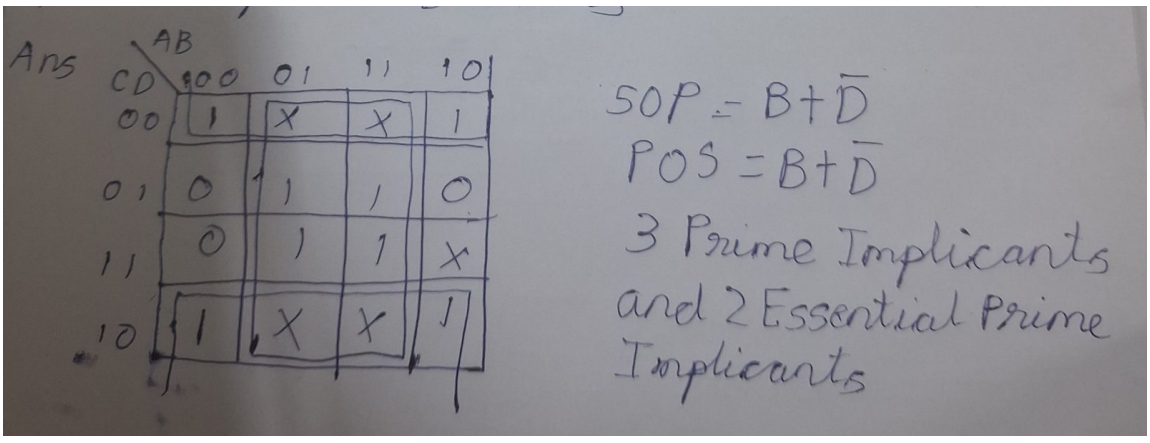


Figure 10.2: Worked solution — Karnaugh map, prime implicants, and minimum SOP/POS expressions.

A second representative example involves a sequential circuit built from two D flip-flops, where the task is to determine the output frequency of Q_2 given an initial state and a known clock frequency. This was solved by tracing the timing diagram of each signal cycle by cycle to identify the repeating pattern of Q_2 relative to the clock.

7. In a given sequential circuit in Fig. 5.7, initial states are $Q1 = 1$ and $Q2 = 0$. For a clock frequency of $1MHz$, find the frequency of signal $Q2$ in kHz (rounded off to the nearest integer). (GATE EC 2023)

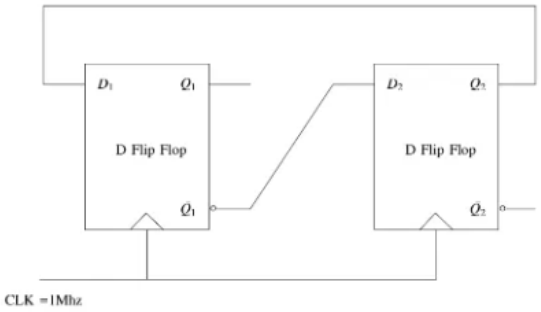


Fig. 5.7

Figure 10.3: GATE EC 2023 — sequential circuit with two D flip-flops; find the frequency of Q_2.

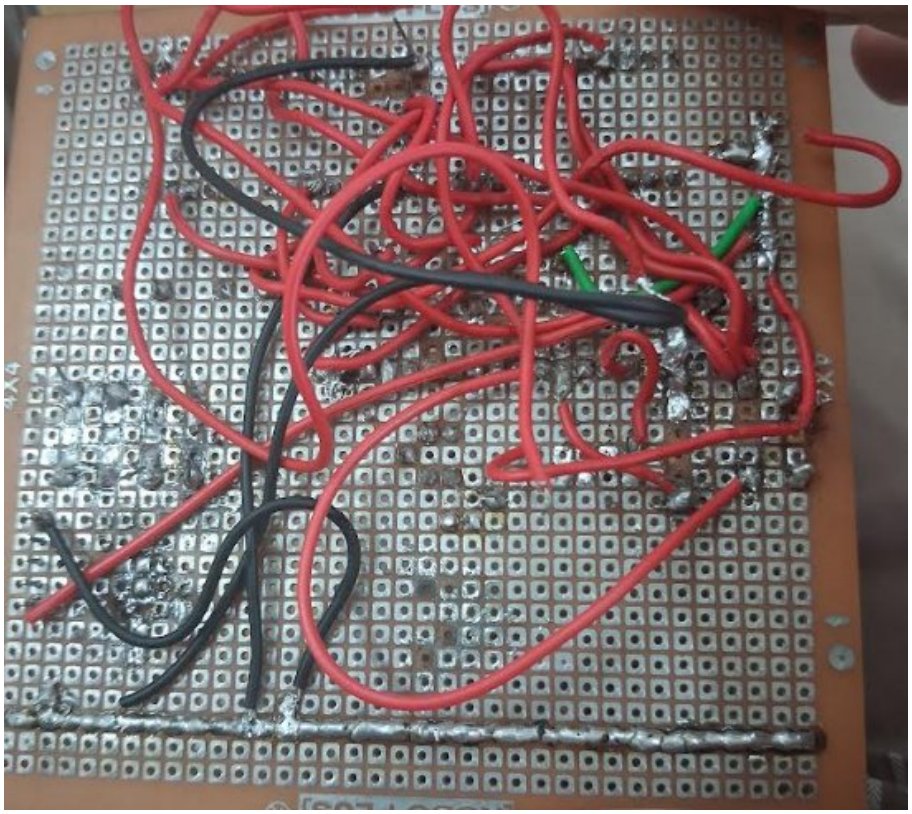


Figure 10.4: Worked solution — timing diagram showing Q_2 repeating every 4 clock cycles, giving $f_{Q_2} = 250\,\text{kHz}$.

Chapter 11

Challenges Faced

Working with real hardware, rather than software alone, meant that several of the biggest challenges during the internship were physical and diagnostic in nature rather than purely programming-related.

11.1 Unconventional Development Workflow

Adapting to a phone-based development workflow using Termux and Arduino Droid, instead of a standard laptop IDE, required extra discipline at each of the three workflow steps. There was no single "Upload" button to fall back on — every compile, file save, and precompiled upload had to be performed correctly and in the right order, and a mistake at any one step (for example, uploading an outdated file from Downloads) could produce confusing results that looked like a hardware fault.

11.2 Short Circuits and Component Damage

Short-circuiting components — including LEDs — occurred at multiple points during breadboard and perfboard work, particularly when wiring density increased in the more complex clock circuit. These incidents occasionally damaged parts outright, requiring them to be replaced and the affected section of the circuit rebuilt and retested from scratch.

11.3 Diagnosing Faults After Soldering

By far the most time-consuming challenge was diagnosing faulty hardware connections after soldering on the perfboard. Unlike a breadboard, where a suspect connection can simply be unplugged and tested, a soldered joint has to be visually inspected, probed with a multimeter, or in some cases desoldered entirely to confirm whether it is the source of a fault. Narrowing down which specific component or connection was not working — versus a fault elsewhere in the circuit or in the uploaded code — often took considerably longer than the soldering itself.

11.4 Bootloader and Fuse Configuration

Ensuring correct fuse settings and successful bootloader burns on standalone ATmega328P chips required careful attention, since a single misconfigured step would leave the chip unable to run any uploaded code at all, with little direct feedback about what had gone

wrong. This meant that failures at this stage often had to be diagnosed by re-checking the entire bootloading procedure rather than by examining the chip itself.

Chapter 12

Learning Outcomes and Reflection

Beyond the specific technical skills listed below, the internship reinforced a broader lesson about embedded systems development: that the gap between code and physical hardware is where most real engineering challenges live. Writing correct code was often the easier half of any given task; making that code run reliably on physical, sometimes imperfect, hand-soldered hardware required patience, systematic debugging, and a willingness to re-check assumptions at every layer, from fuse bits to cold solder joints.

Working through the three-step workflow — hardware, software-level code, and assembly-level code — for the same tasks also gave a much more concrete understanding of what a conventional Arduino IDE normally does automatically. Having to compile, generate hex files, and upload precompiled binaries manually made the underlying embedded toolchain visible rather than hidden, which is knowledge that carries over to working with other microcontroller platforms in the future.

Chapter 13

Skills Gained

- Embedded C/C++ programming and assembly-level programming for AVR microcontrollers.
- Practical understanding of the compilation and flashing pipeline: source code → compiled binary/hex file → hardware upload.
- Command-line proficiency using Linux and Termux for embedded development.
- Working knowledge of digital ICs (7447, 7474) and their role in display and counter circuits.
- Bootloader burning and standalone microcontroller programming using Arduino as an ISP.
- Circuit prototyping on breadboards and permanent circuit assembly through soldering on perfboards.
- Hardware debugging — systematically isolating faulty components and connections in both prototype and soldered circuits.
- Learnt several key concepts of Digital Electronics related to Latches, Flip Flops (SR, JK, D, T).
- Learnt the basics about Finite State Machine (FSM).
- Learnt to draw the timing diagrams of waveforms of digital signals in a circuit, cycle by cycle.

Chapter 14

Conclusion

This internship at the Electrical Engineering Core Labs, IIT Hyderabad, provided substantial hands-on exposure to embedded systems development, from low-level assembly programming to physical circuit assembly. Working through the complete pipeline — hardware design, software-level coding, and assembly-level coding — on real hardware, using an unconventional phone-based toolchain, offered a far deeper understanding of how embedded systems function than a typical IDE-based workflow would have. The culmination of this experience in an independently built, soldered digital clock, built around a standalone, bootloader-programmed ATmega328P chip, demonstrated the practical application of every concept covered during the internship. The debugging challenges encountered along the way, particularly during the soldering and testing phases, strengthened problem-solving skills that are broadly applicable to embedded hardware work.

I am grateful to Prof. G.V.V. and the Electrical Engineering Core Labs at IIT Hyderabad for the opportunity, guidance, and resources provided throughout this internship.

Appendix A

Glossary of Key Terms

The following terms, specific to the tools and workflow used during this internship, are summarized here for reference.

Termux A terminal emulator and Linux environment app for Android, used here to write and compile embedded code directly on a smartphone.

ArduinoDroid An Android application that can upload code to Arduino boards; used specifically in "Upload Precompiled" mode to flash binaries and hex files compiled in Termux.

USB OTG USB On-The-Go — a cable/adapter standard that allows a smartphone to act as a USB host, enabling a direct wired connection between the phone and the Arduino Uno.

Bootloader A small pre-installed program on a microcontroller that allows new code to be uploaded to it without a separate hardware programmer; "burning" a bootloader installs this program and sets the associated fuse bits.

Fuse Bits Configuration bits on an AVR microcontroller (such as the ATmega328P) that determine low-level hardware behavior, including clock source and bootloader presence.

ISP (In-System Programmer) A method of programming a microcontroller directly, without going through its bootloader; here, an Arduino Uno was configured to act as an ISP for standalone ATmega328P chips.

ATmega328P The AVR microcontroller chip used on the Arduino Uno board; in this internship, standalone versions of this chip were bootloaded and programmed independently of the Uno board.

Hex File A file format (Intel HEX) containing compiled machine code in a form that can be uploaded directly onto a microcontroller's program memory.

Perfboard A rigid board with a grid of pre-drilled holes and copper pads, used for permanently soldering and wiring together electronic components, as opposed to a reusable breadboard.

Appendix B

References and Resources

- Setup and package installation instructions provided by Prof. G.V.V., Department of Electrical Engineering, IIT Hyderabad (shared via GitHub).
- Termux documentation, for Linux environment setup and package management on Android.
- ArduinoDroid application documentation, for the “Upload Precompiled” workflow.
- Datasheets for the 7447 BCD-to-seven-segment decoder and 7474 dual D-type flip-flop ICs.
- ATmega328P microcontroller datasheet, for fuse bit configuration and bootloader specifications.
- Morris Mano, *Digital Design*.